

Documentación del Componente: LoginModal

1. Descripción General

El componente **LoginModal** es una ventana modal desarrollada en React que permite a los usuarios autenticarse o registrarse dentro del sistema.

Este componente gestiona dos procesos principales:

- Inicio de sesión de usuarios existentes.
- Registro de nuevos usuarios.

Además, proporciona validaciones básicas, manejo de errores y retroalimentación visual mediante SweetAlert2.

2. Objetivo

Permitir que los usuarios accedan al sistema de manera segura mediante autenticación local y que los nuevos usuarios puedan crear una cuenta desde la misma interfaz.

3. Funcionalidades Principales

3.1 Inicio de Sesión

El usuario ingresa:

- Correo electrónico
- Contraseña

El sistema envía las credenciales al servidor mediante una petición HTTP POST.

Si las credenciales son válidas:

- Se recibe un token de autenticación.
- Se almacena la sesión en LocalStorage.
- Se guarda la información básica del usuario.
- Se ejecuta el proceso de login en la aplicación.

3.2 Registro de Usuarios

Permite crear una nueva cuenta solicitando:

- Nombre completo
- Correo electrónico
- Contraseña
- Confirmación de contraseña

Antes de enviar la información se valida que ambas contraseñas coincidan.

Posteriormente se realiza una petición HTTP POST al servidor para registrar al usuario.

3.3 Validación de Contraseñas

Durante el registro:

- La contraseña debe coincidir con el campo de confirmación.
 - Si no coinciden, se muestra un mensaje de error y se cancela el proceso.
-

3.4 Manejo de Sesión

Después de un inicio de sesión exitoso se almacenan:

Token de autenticación

token

al_token

Información del usuario

al_usuario

Incluye:

- ID
 - Nombre
 - Correo electrónico
 - Rol
 - Avatar
-

4. Tecnologías Utilizadas

Tecnología	Propósito
React	Construcción de la interfaz
useState	Gestión de estados
Fetch API	Comunicación con el servidor
SweetAlert2	Mensajes interactivos
Bootstrap	Diseño visual

5. Estados del Componente

Estado	Descripción
isLogin	Define si se muestra login o registro
nombre	Nombre del usuario
email	Correo electrónico
password	Contraseña
confirmar	Confirmación de contraseña

6. Flujo de Inicio de Sesión

Paso 1

El usuario abre el modal.

Paso 2

Selecciona la opción Ingresar.

Paso 3

Ingresa correo y contraseña.

Paso 4

El sistema envía las credenciales al servidor.

Paso 5

El servidor valida la información.

Paso 6

Si la autenticación es correcta:

- Se almacena el token.
- Se guarda la información del usuario.
- Se cierra el modal.
- Se muestra mensaje de bienvenida.

Paso 7

La aplicación actualiza el estado de sesión.

7. Flujo de Registro

Paso 1

El usuario selecciona Registrarse.

Paso 2

Completa:

- Nombre
- Correo
- Contraseña
- Confirmación

Paso 3

El sistema verifica que las contraseñas coincidan.

Paso 4

Se envían los datos al servidor.

Paso 5

El servidor crea el usuario con rol Cliente.

Paso 6

Se muestra mensaje de éxito.

Paso 7

El sistema redirige automáticamente al formulario de inicio de sesión.

8. Manejo de Errores

Error de autenticación

Se presenta cuando:

- El correo no existe.
- La contraseña es incorrecta.

Respuesta:

Acceso denegado

Correo o contraseña incorrectos

Error de registro

Se presenta cuando:

- El correo ya existe.
- El servidor rechaza la creación.

Respuesta:

No se pudo registrar

Error de conexión

Se presenta cuando:

- El servidor no está disponible.
- Existe un problema de red.

Respuesta:

No se pudo conectar con el servidor central.

9. Integración con Backend

Endpoint de Login

POST /login-local

Datos enviados

```
{  
  "email": "usuario@correo.com",  
  "password": "123456"  
}
```

Endpoint de Registro

POST /usuarios

Datos enviados

```
{  
  "usuario": "Juan Perez",  
  "correo": "juan@correo.com",  
  "password_hash": "123456",  
  "rol": "cliente"  
}
```

10. Requerimientos Funcionales

RF-01

El sistema debe permitir iniciar sesión mediante correo y contraseña.

RF-02

El sistema debe permitir registrar nuevos usuarios.

RF-03

El sistema debe validar la coincidencia de contraseñas.

RF-04

El sistema debe almacenar la sesión del usuario.

RF-05

El sistema debe informar los errores de autenticación y conexión.

11. Requerimientos No Funcionales

RNF-01

La interfaz debe ser intuitiva y responsiva.

RNF-02

Las respuestas visuales deben mostrarse mediante ventanas emergentes.

RNF-03

La autenticación debe realizarse mediante peticiones HTTPS en producción.

RNF-04

La sesión debe persistir mediante almacenamiento local.

12. Resultado Esperado

El componente LoginModal proporciona una solución centralizada para la autenticación y registro de usuarios, garantizando una experiencia de acceso sencilla, segura y amigable dentro de la plataforma de comercio electrónico.

```
import { useState } from "react";
import Swal from "sweetalert2";

function LoginModal({ onClose, login }) {
  const [isLogin, setIsLogin] = useState(true);
  const [nombre, setNombre] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmar, setConfirmar] = useState("");

  const manejarEnvio = async (e) => {
    e.preventDefault();

    try {
      if (isLogin) {
        // 1. PETICIÓN DE LOGIN AL BACKEND DE NODE.JS
        const response = await
fetch("http://localhost:3001/login-local", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            email: email.trim(),
            password: password.trim(),
          }),
        });

        const data = await response.json();

        if (!response.ok || !data.success) {
          Swal.fire({
            icon: "error",
            title: "Acceso denegado",
            text: data.message || "Correo o contraseña incorrectos",
            confirmButtonColor: "#10142D"
          });
          return;
        }

        // Guardamos el token en el localStorage por si necesitas
proteger rutas después
        if (data.token) {
          localStorage.setItem("token", data.token);
          localStorage.setItem("al_token", data.token);
        }
      }
    }
  }
}
```



```

    }

    const usuario = {
      id: data.usuario.id,
      nombre: data.usuario.nombre,
      email: data.usuario.email,
      rol: data.usuario.rol,
      avatar_url: data.usuario.avatar_url || null
    };
    localStorage.setItem("al_usuario", JSON.stringify(usuario));

    onClose();
    Swal.fire({
      icon: "success",
      title: "¡Bienvenido de nuevo!",
      text: `Hola, ${usuario.nombre}`,
      confirmButtonColor: "#F5A623",
      timer: 2000,
      showConfirmButton: false,
    }).then(() => login(usuario));

  } else {
    // 2. PETICIÓN DE REGISTRO AL BACKEND DE NODE.JS
    if (password !== confirmar) {
      Swal.fire({ icon: "error", title: "Las contraseñas no coinciden" });
      return;
    }

    const response = await fetch("http://localhost:3001/usuarios",
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        usuario: nombre.trim() || email.trim().split("@")[0],
        correo: email.trim(),
        password_hash: password.trim(), // El backend recibe la
clave e inicializa como 'cliente'
        rol: "cliente"
      }),
    });

    const data = await response.json();

```

```

        if (!response.ok) {
            Swal.fire({
                icon: "error",
                title: "No se pudo registrar",
                text: data.error || "Ocurrió un error inesperado al crear
la cuenta",
                confirmButtonColor: "#10142D"
            });
            return;
        }

        Swal.fire({
            icon: "success",
            title: "Cuenta creada",
            text: "Tu registro se completó con éxito. Ya puedes iniciar
sesión.",
            confirmButtonColor: "#F5A623"
        }).then(() => {
            setIsLogin(true); // Lo mandamos al login para que ingrese
con sus datos
            setPassword("");
            setConfirmar("");
        });
    }
} catch (error) {
    console.error("Error:", error);
    Swal.fire({
        icon: "error",
        title: "Error de red",
        text: "No se pudo conectar con el servidor central.",
        confirmButtonColor: "#10142D"
    });
}
};

return (
    <div
        style={{
            position: "fixed", inset: 0,
            backgroundColor: "rgba(16, 20, 45, 0.7)",
            backdropFilter: "blur(8px)",
            zIndex: 9998,

```



```

        });
        }, 100);
    }}
    >
    
    <span className="fw-bold" style={{ fontSize: "0.9rem"
}}>Continuar con Google</span>
    </button>

    <div className="d-flex align-items-center my-4">
        <hr className="flex-grow-1 text-muted opacity-25" />
        <span className="px-3 text-muted small fw-bold">O CON TU
CORREO</span>
        <hr className="flex-grow-1 text-muted opacity-25" />
    </div>

    <div className="bg-light rounded-pill p-1 d-flex mb-4"
style={{ border: "1px solid #eee" }}>
        <button
            className={`flex-fill border-0 rounded-pill py-2 fw-bold
transition-all ${isLogin ? "bg-white shadow-sm text-warning" :
"bg-transparent text-secondary"}`}
            style={{ fontSize: "0.8rem" }}
            onClick={() => setIsLogin(true)}
        >
            INGRESAR
        </button>
        <button
            className={`flex-fill border-0 rounded-pill py-2 fw-bold
transition-all ${!isLogin ? "bg-white shadow-sm text-warning" :
"bg-transparent text-secondary"}`}
            style={{ fontSize: "0.8rem" }}
            onClick={() => setIsLogin(false)}
        >
            REGISTRARSE
        </button>
    </div>

    <form onSubmit={manejarEnvio}>
        <div className="form-floating mb-3">

```

```

        <input type="text" className="form-control border-0
bg-light shadow-none"
        placeholder="Nombre" value={nombre} onChange={(e) =>
setNombre(e.target.value)} required />
        <label className="text-muted small">Nombre
Completo</label>
    </div>
    )}

    <div className="form-floating mb-3">
        <input type="email" className="form-control border-0
bg-light shadow-none"
        placeholder="Email" value={email} onChange={(e) =>
setEmail(e.target.value)} required />
        <label className="text-muted small">Correo
Electrónico</label>
    </div>

    <div className="form-floating mb-3">
        <input type="password" className="form-control border-0
bg-light shadow-none"
        placeholder="Contraseña" value={password} onChange={(e)
=> setPassword(e.target.value)} required />
        <label className="text-muted small">Contraseña</label>
    </div>

    {!isLogin && (
        <div className="form-floating mb-3">
            <input type="password" className="form-control border-0
bg-light shadow-none"
            placeholder="Repetir" value={confirmar} onChange={(e)
=> setConfirmar(e.target.value)} required />
            <label className="text-muted small">Confirmar
Contraseña</label>
        </div>
    )}

    <button type="submit" className="btn btn-warning w-100 py-3
fw-bold mt-2 shadow-sm"
        style={{ borderRadius: "14px", letterSpacing: "1px" }}>
        {isLogin ? "INICIAR SESIÓN" : "CREAR CUENTA GRATIS"}
    </button>
</form>

```

```
        </div>
      </div>
    </div>
  );
}

export default LoginModal;
```